Annals of Computer Science and Intelligence Systems
https://doi.org/10.5281/zenodo.19186370

A Framework for Generalized Risk-Aware Planning and Decision-Making under Dynamic Adversarial Conditions

Aditi Srivasatava[1], Krishna Agarwal[2], Mahesh Kumar Tiwari[3], Rinku Raheja[4], Shweta Sinha[5], Shalini Lamba[6]

[1, 2] Research Scholar, National PG College, Lucknow University
[3, 4, 5,6]Assistant Professor, National PG College, Lucknow University
[4 *] Corresponding Author: Rinku Raheja

Received: Feb 14, 2026 **Accepted:** Mar 14, 2026 **Published:** Mar 23, 2026

Abstract:

Path planning in dynamic and adversarial environments has been one of the most crucial challenge in the world of artificial intelligence and robotics. Traditional algorithms such as A* and ARA* primarily aimed at achieving the optimized path often neglecting the adversarial threats and safety concerns. Addressing this limitation, in this paper we have proposed the ARA++, a novel framework that extends the ARA* algorithm by integrating probabilistic enemy risk factor modeling, adaptive thresholding and multi-objective cost functions. The ARA++ algorithm unlike conventional path finding algorithms evaluates risk factor and survival probability for nodes at each time interval providing dynamic updates of the risk factors on the basis of adversary movement. A safety threshold value must be defined for the problem such that if any path's survival probability value falls below the safety threshold, the path is pruned so that only safe trajectories are considered. A case study of 3*3 maze problem is considered that clearly demonstrates the working of algorithm and finding the shorter as well as safer path maximizing the probability of survival. Experimental analysis confirms that ARA++ outperforms the conventional algorithms A* and ARA* in adversarial settings by producing safer, cheap, adaptive and computationally efficient paths. Extending the limits from a simple maze problem, the algorithm is generalized to be used across different fields that can be automated vehicles, robotics and cybersecurity, thus, making it an entire framework.

Keywords: Minimax Problem, Alpha Beta Pruning, A*, Risk Aware A*, ARA++

1. Introduction:

Artificial Intelligence (AI) is a rapid evolving field of Computer Science that aims build machines having human intelligence and ability for logical thinking, recognizing, reasoning and memorizing. Artificial Intelligence has not only evolved in the IT sector but also emerged in different other sectors including healthcare, financial, educational, agriculture, defense, gaming and many more. In gaming sector, artificial intelligence has evolved for its non-player character (NPC) behavior where it simulates a human player that has thinking and decision-making ability to make the best possible move for winning the game. Unlike hardcoded game applications, AI enables applications to work dynamically making it more challenging and engaging.

The game playing algorithms have been classified as game tree search algorithms and pathfinding algorithms. The strategic competitive games such as tic-tac-toe and chess widely make use of different game tree search algorithms such as minimax problem and alpha beta pruning whereas path finding games like maze problem uses different path finding algorithms

like A* algorithm, AO* algorithm, Best First Search and many more. The decision of the choice of algorithm is taken as per the requirements.

Modern games such as maze problem with an enemy aroused the need of integration of both types of game playing algorithms i.e. game tree search algorithms and pathfinding algorithms. Thus, a hybrid approach of game tree search and pathfinding algorithms can thus offer significant improvements in game AI, providing both smarter decision making and faster pathfinding capabilities to the AI agent.

Apart from game mechanics, AI also drives procedural content generation, analysis of player behavior, and adaptive difficulty adjustment, making the games more immersive and personalized.

2. Literature Review

Research	Objective	Algorithm	Conclusion	Future Scope
Junkang Li,	To make an algorithm for the games which have incomplete information.	AND-OR and Alpha Beta Pruning	By the combination of alpha beta lower and upper bound values the algorithm becomes more efficient in terms of visiting the nodes.	Optimization to be performed in alpha beta search in games along with the cache applied to it
Fatima Sohail Shaukat, 2022	To classify various searching techniques in AI based on its optimality.	Informed Seach and Uninformed Search algos	We are able to identify which algo is performing best at each step.	Time complexity can be reduced.
Ka Heng Chan	To find the shortest path of a rectangular maze and comparing the distance using Manhattan and Euclidean distance formulas.	A* algorithm	Manhattan distance gives a better result in comparison to Euclidean distance on the basis of time consumed	To enhance the efficiency of A* algorithm by accommodating multi agent scenario.
Kaili Xie	To improve efficiency of the trajectory search for flights ensuring safety of aircraft.	D* Lite, improved D* Lite	The planned trajectory remains away from obstacles providing safety along with efficiency.	More planning is required for local environment.
Rijul Nasa, 2018	To optimize the move in a connect-4 game.	Mini-max and Alpha Beta Pruning	Time complexity is reduced by	More optimization can be achieved.

			combining both the algorithm than only using mini-max algorithm.	

3. Background

3.1. Minimax Algorithm

The minimax algorithm is a fundamental game tree search algorithm used for decision making in the two player games such as chess, tic-tac-toe and checkers where one player's profit results in the loss of another player and vice versa.

The minimax algorithm works on the principle of minimizing and maximizing the possibility of winning i.e. the game tree is divided in two types of nodes –

- Maximizer (Max): Tries to maximize the score for Player A
- Minimizer (Min): Tries to minimize the score for Player A i.e. maximize the score for Player B

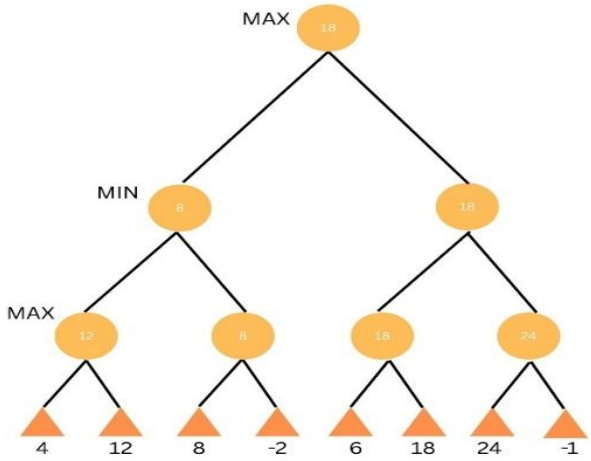

Fig. 3.1 – Working of Minimax Algorithm with heuristic values

The evaluation function i.e. the game result is stored at the leaf nodes of the tree which is propagated upwards by the maximizer (maximum heuristic value is chosen) and minimizer (minimum heuristic value is chosen) nodes to the root node which helps to decide the most optimal move at the root node which maximizes the score.

The time complexity of the minimax algorithm is $O(b^d)$ where b is the branching factor and d is the depth of the game tree.

$$Minimax(n) = \begin{cases} Utility(n), if n\ is\ a\ terminal\ node \\ maxMinimax(c), if\ n\ is\ a\ maximizer \\ minMinimax(c), if\ n\ is\ a\ minimizer \end{cases}$$

where Utility(n) is the evaluation function at the terminal nodes and c are the children of node n.

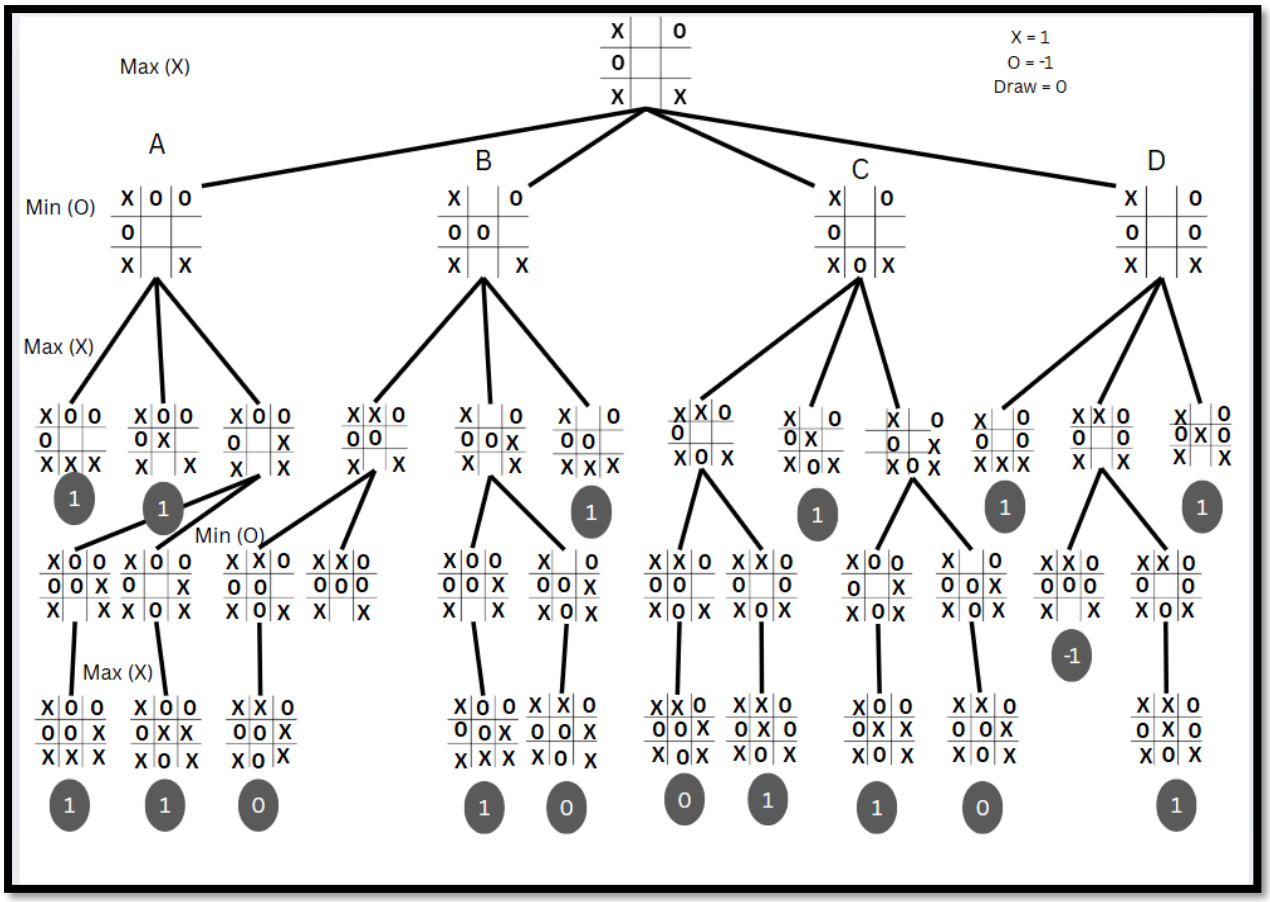

Fig. 3.2 Minimax problem in a tic-tac-toe game

Consider the Fig. 3.2 in which the game of tic-tac-toe is shown to depict the minimax algorithm. The maximizer nodes in the tree depict the × move whereas the minimizer nodes depict the O move. The heuristic value of the × is taken as 1, for O is taken as -1 and for draw it is taken as 0.

At the root node, player X (Maximizer) has four possible moves: A, B, C, and D.

- If player X chooses the branch with heuristic value = 1, it will eventually lead to a winning state for X.
- Suppose X initially considers move B. Then it is O's turn (Minimizer). Player O will try to reduce X's chances of winning by choosing the child node with the minimum value.

3.2. Alpha Beta Pruning

Alpha Beta Pruning is an optimization of minimax algorithm used to reduce the time complexity of the minimax algorithm by eliminating (pruning) the nodes of the tree that are unlikely to give the better move or unlikely to influence the current decision so that they are not explored. Like minimax algorithm, this technique is basically used in two player games such as chess, tic-tac-toe and checkers.

Alpha beta pruning works on the two parameters – alpha (α) and beta (β) where alpha denotes the best value that the maximizer can guarantee so far whereas beta denotes the best value that the minimizer can guarantee so far. The alpha beta works on the following principles –

- At maximizers, α is updated if greater heuristic value is found
- At minimizers, β is updated if smaller heuristic value is found
- At maximizers, if $\alpha \geq \beta$, further nodes are pruned because β will always minimize the value which will never produce any value greater than α, thus would not affect the value of α

- At minimizers, if β ≤ α, further nodes are pruned because α will always maximize the value which will never produce any value lesser than β, thus would not affect the value of β

The time complexity of the minimax algorithm in best case is $O(b^{d/2})$ where b is the branching factor and d is the depth of the game tree. However, in case of worst case, it can still be $O(b^d)$ where no nodes are pruned.

$$AlphaBeta(n,\alpha,\beta) = \begin{cases} Utility(n), f(n)\ is\ a\ terminal\ node \\ maxAlphaBeta(c,\alpha,\beta), if\ n\ is\ a\ maximizer\ with\ \alpha = max(\alpha, value) \\ minAlphaBeta(c,\alpha,\beta), if\ n\ is\ a\ minimizer\ with\ with\ \beta = min(\beta, value) \end{cases}$$

In Fig. 3.3 pruning occurs at the following nodes

- At maximizer node, the value of α is 32 i.e. the final value of α here must be ≥ 32. However, its next child beta node has value 15 i.e. it will be ≤ 15. Thus α > β and a value ≥ 32 and ≤ 15 is impossible. Therefore node 16 is pruned
- At maximizer node, the value of α is 17 i.e. the final value of α here must be ≥ 17. However, its next child beta node has value -11 i.e. it will be ≤ -11. Thus α > β and a value ≥ 17 and ≤ -11 is impossible. Therefore node 22 is pruned
- At minimizer node, the value of β is 17 i.e. the final value of β here must be ≤ 17. However, its next child alpha node has value 23 i.e. it will be ≥ 23. Thus, β > α and a value ≤ 17 and ≥ 23 is impossible. Therefore nodes 22 and 65 are pruned

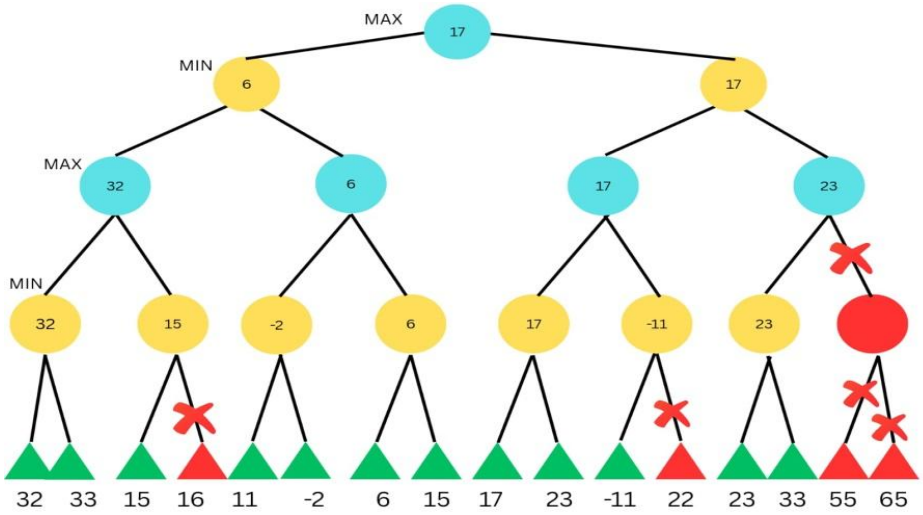

Fig. 3.3 - Working of alpha beta pruning with heuristic values

3.3. A* Algorithm

A* algorithm is an informed (heuristic) search algorithm used in the field of artificial intelligence for finding the optimal path from source node to the destination node. The algorithm was first introduced by Hart, Nilsson, and Raphael in 1968. The algorithm is aimed to be a complete algorithm that guarantee the optimality of the chosen path in case of admissible heuristic, thus it has found great potential in the field of game development, robotics navigation and network optimization. Unlike the old traditional algorithms such as Best First Search which considered only the heuristics, the A* not only considers the heuristic value but also the present value making it a complete and optimal algorithm

The algorithm obeys the formula $f(n) = g(n) + h(n)$ where:

f(n): Evaluation function considering actual cost from source node to the current node and estimated (heuristic) cost from current node to the goal node

g(n): Actual cost from the source node to the current node
h(n): Estimated (heuristic) cost from the current node to the goal node
The commonly used heuristics are Manhattan distance (for grid-based movement), Euclidean distance (for diagonal or straight-line movement) and Chebyshev distance (for 8 direction movement).

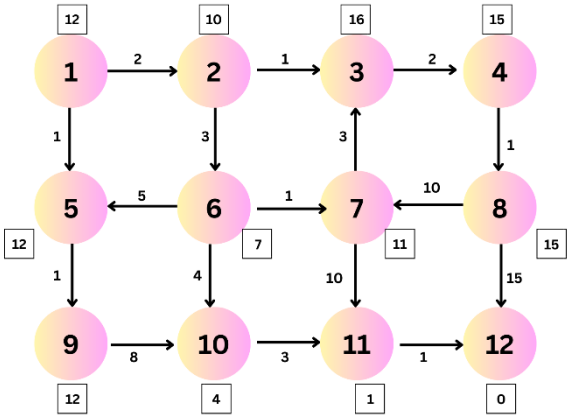

Fig. 3.4 – A* Problem with f(n) and h(n) values

The Fig. 3.4 portrays the A* problem where the actual costs from one node to another are represented as edge weights and the heuristic costs (h(n)) from the current node to the goal node are shown in the squares. The heuristic distance from node 1 to 12 is underestimated as 12 units and later in the state space tree as shown in the Fig. 3.5, the minimum cost taken to reach node 12 from node 1 is calculated as 13 units.

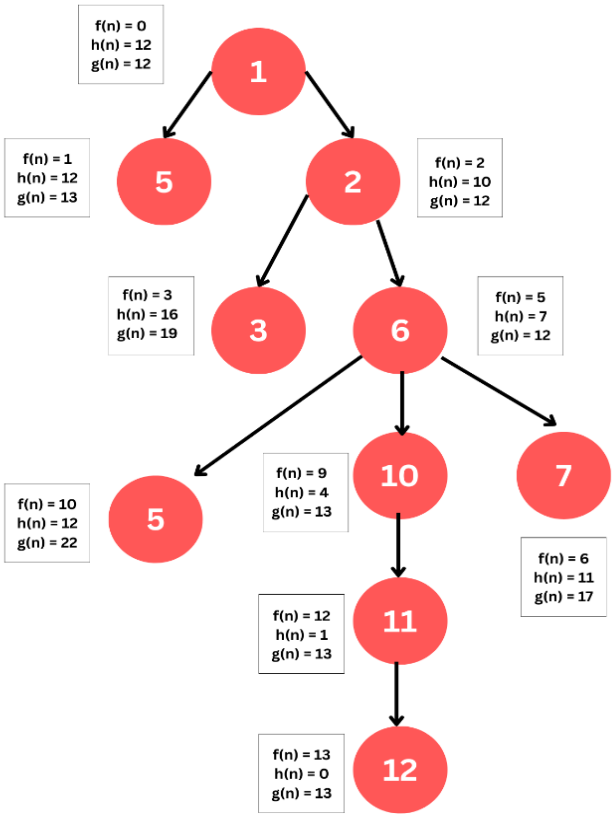

Fig. 3.5 – Solution Tree of problem in Fig. 4

A* is known as admissible as it guarantees to give the optimal solution depending on the heuristics. For A* to give the optimal solution, the heuristic choice must be underestimated

and not overestimated i.e. h(n) ≤ h*(n) where h(n) is the heuristic cost from the current node to the goal node whereas h*(n) is the actual cost from the current node to the goal node. In case of overestimation, A* may choose a suboptimal path due to overestimated cost.

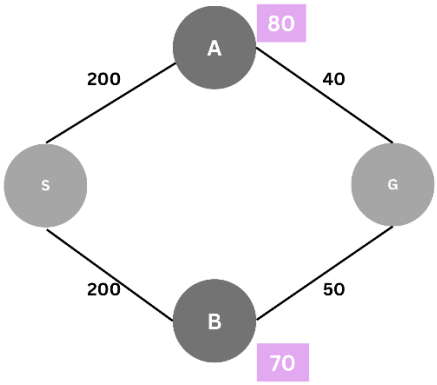

Fig. 3.6 – Admissibility of A* Algorithm

Consider the Fig. 3.6 where S is the source node and G is the goal node. In case of overestimation, g(n) = f(n)+h(n) = 200 + 70 of node B is less as compared to that of node A, so path S->B->G is chosen and then this path cost (250) is less than g(n) at node A i.e. 200+80=280. So, A* stops and gives S->B->G as optimal solution, however the optimal solution was S->A->G. However, if cost would have been underestimated at node (suppose 30), the algorithm would also have explored that path and chosen it as the optimal path.

3.4. Bayesian Inference

It is a statistical procedure for decision making in the face of uncertainty. It uses Bayes' Theorem,

where probability of event B is computed given the envent A has already occur. The prior belief is updated using the Bayes Theorem and produces the posterior probability distribution. It's a common approach in artificial intelligence for estimating the hidden variable, reducing uncertainty to make more accurate predictions. In terms of search algorithm, the method can be employed to judge the quality of heuristics. That is to say that we can represent the behavior of opponents in an adversarial problem. Using this in hybrid of Alpha Beta and A* we can (probably) allow system to change and adapt, as soon as there are uncertain stuff, make in time decisions which may further require the system to zigzag even at decisions that appear best at first sight.

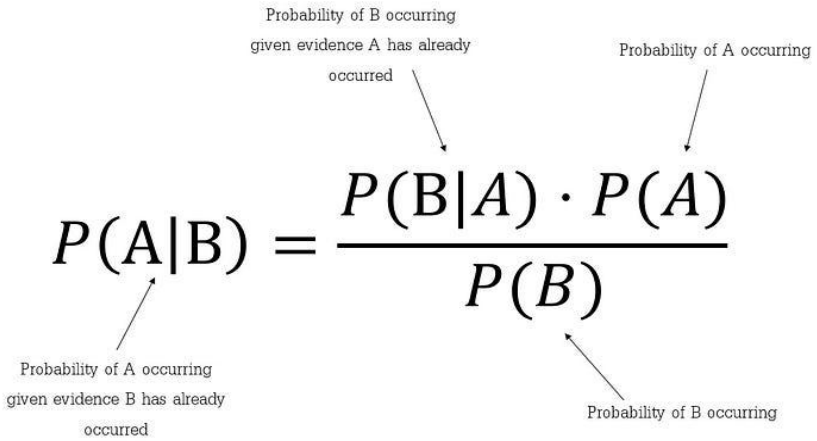

Fig. 3.7 – Bayesian Inference Formula

3.5. Monte Carlo Simulation

It is the computational technique which is based on random sampling to approximate the solution of mathematical problems. To calculate the expectations of a function f(x) over a random variable x Monte Carlo uses:

$$E[f(x)] \approx N1 \sum i = 1Nf(xi)$$

Where N is the number of random samples and xi is independent sample drawn from some domain. This approach can be used in Artificial Intelligence for evaluating game sates in games like Monte Carlo Tree Search and in path finding during the time of uncertainty. Our hybrid algorithm can become more reliable with the help of this algorithm because we can approximate the expected results at leaf node in stochastic environment.

3.6. Risk Aware A* Algorithm

It is the advance version of traditional A* heuristic search algorithm that not only accounts for the shortest path but also take cares to choose the safest node possible. This algorithm has additional feature of risk management like collision probability, finding uncertainty in edge nodes and worst case performance. It helps in solving the real world problems like in field of robotics and autonomous navigation where safety is equally important as efficiency. Including this algorithm in our hybrid approach will ensure to identify paths that that minimizes potential failures making it suitable for critical ready tasks.

3.7. D* Algorithm

D* which stands for dynamic A* algorithm which is made to find the path of the problem that changes over time. D* repairs the existing path to reflect environmental changes which is better than A* which re-plans from scratch whenever an obstacle is detected. This algorithm is beneficial in adaptive AI Systems where agents must continuously navigate through dynamic terrains. Integrating this in our hybrid approach would make the model capable to re-plan the path dynamically making it closer to solve non static scenarios.

4. Methodology

4.1 Overview

The proposed algorithm, Adaptive Risk Aware A++ (ARA++) is the extension of traditional A* and Risk Aware A* algorithms with a taste of Monte Carlo Simulation, Bayesian inference, D*, Alpha Beta pruning and D* Lite algorithms that is not only limited to the universe of gaming but can solve real world problems by integrating incremental replanning, probabilistic enemy modeling, and multi-objective optimization. The key idea of the algorithm is to balance the path optimality, survival probability, and real-time adaptability in dynamic and uncertain environments.

4.2 Key Features of ARA++

Table 4.1 – Features of ARA++ Algorithm

Feature	Description
Incremental Planning	Reduces computational overhead by reusing prior search results (inspired by D* Lite).
Probabilistic Enemy Modeling	Predict adaptive enemy strategies (inspired by Bayesian inference).
Multi-Objective Cost Function	Integration of path length, survival probability, and path deviation penalties.
Adaptive Thresholding	Dynamically adjusts survival probability thresholds.

Approximate Multi-Enemy Handling	Manages multiple enemies via independent approximation or Monte Carlo sampling.
Anytime Search	Quickly provides a feasible path and refines it as computation time allows.

4.3 Algorithm Methodology

Step 1: Identify the problem

- Environment: A map or graph in which the nodes can be traversed.
- Start Node (s): The position of the agent at time t=0
- Goal Node (g): The desired destination for the agent.
- Enemies Model: each enemy e will move probabilistically with a known policy, or a patrolling path.
- Objective: Minimize a multi-objective cost function that is composed of penalties for distance to the goal, risk of getting intercepted, and penalties for deviating from the true shortest path.

Step 2: Enemy Behavior Modeling

For each enemy e:

- Chasing Probability (p_c): Enemy tries to chase the agent or moves towards the agent.
- Patrol Probability (p_p): Enemy patrols over a predefined path.
- Move Probability:

$$P(m|e,t) = \begin{cases} pc & if\ enemy\ move\ m\ is\ towards\ player \\ pp & if\ move\ m\ is\ along\ patrol\ path \end{cases}$$

Where m is a possible move at time t.

- Threat Function (δ (n, m)):

$$\delta(n,m) = \begin{cases} 1 & if\ enemy\ move\ m\ threatens\ node\ n \\ 0 & if\ enemy\ move\ m\ does\ not\ threatens\ node\ n \end{cases}$$

Step 3: Multi-Objective Cost Function

For each candidate node n at time t, ARA++ computes:

$$g(n,t) = f(n) + h(n) + \lambda \cdot R(n,t) + \mu \cdot D(n)$$

Risk Computation:

$$R(n,t) = 1 - e \prod \left(1 - m \in M \sum P(m \mid e,t) \cdot \delta(n,m)\right)$$

Step 4: Candidate Node Expansion

1. For the current node n, generate all adjacent or reachable nodes n′.
2. Compute cost g(n′, t+1) for each candidate node using Step 3.
3. Insert each candidate node into a priority queue ordered by g(n′, t+1).

Step 5: Anytime Planning and Incremental Updates

- ARA++ generates an initial feasible path quickly (anytime property).
- When new enemy information or map updates arrive:
  - Update risk R (n, t) for affected nodes.
  - Recompute g (n, t) only for impacted nodes (incremental repair).
  - Update priority queue efficiently instead of replanning from scratch.

Step 6: Path Selection Strategy

- At each step, the algorithm selects the candidate node with minimum expected cost g (n, t).
- Trade-off: ARA++ balances between the shortest path (low f(n)+h(n)) and low-risk path (low R (n, t)) on the basis of λ.
- If multiple paths have similar g (n, t), the algorithm prioritizes the paths with lower deviation D(n).

Step 7: Algorithm Termination

- The algorithm terminates when the goal node g is reached.
- The output path is risk-aware, possibly longer than the naive shortest path but safer against enemy interception.

4.4 Mathematical Formulation

Let a path P = $\{n_0, n_1, \ldots, n_k\}$ from start node n_0 to goal node n_k

4.4.1 Multi Objective Cost Function

$$g(n,t) = f(n) + h(n) + \lambda \cdot R(n,t) + \mu \cdot D(n)$$

Table 4.2 – Notations used in Multi Objective Cost Function

Notation	Description
f(n)	Cost function from start node to node n
h(n)	Heuristic estimate from node n to goal node
R(n,t)	The risk of being intercepted by enemy at time t
D(n)	Path deviation penalty (encourages smoother paths)
λ, μ	Adaptive weights, tuned based on environment conditions

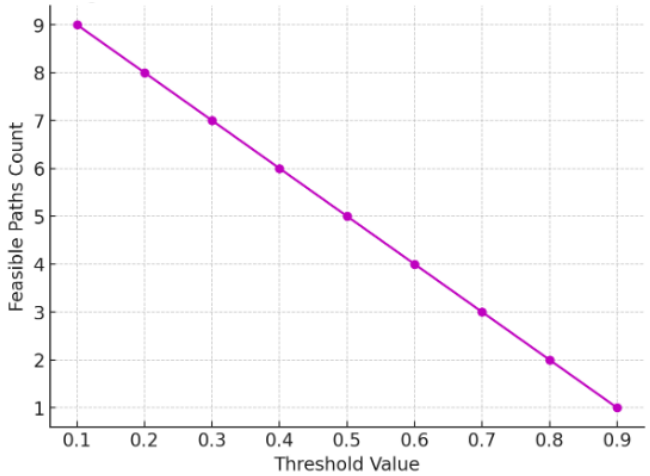

Fig. 4.1 – Effect of threshold on Feasible paths count

Table 4.3 – Definition of parameters used in the algorithm

Notation	Meaning	Adaptation Strategy
Λ	Weight for risk term	Increased when enemy uncertainty is high
M	Weight controlling how strongly deviation affects the path choice	Increased in constrained environments
T	Adaptive risk threshold	Dynamically lowered when few feasible paths exist

4.4.2 Enemy Risk Model

For enemy at position e with possible moves M = $\{m_1, m_2, \ldots, m_j\}$:

$$R(n,t) = 1 - e \prod \left(1 - m \in M \sum P(m \mid e,t) \cdot \delta(n,m)\right)$$

Table 4.4 –Notations used in the Enemy Risk Model

Notation	Description
P(m|e,t)	Probability that enemy e chooses move m at time t
δ(n,m)	Indicator Function (1 if move m threatens node n, else 0)

4.4.3 Bayesian Updating

$$P(Hi \mid Ot) = \frac{P(Ot \mid Hi) \cdot P(Hi)}{\sum jP(Ot \mid Hj) \cdot P(Hj)}$$

Table 4.5 – Notations used in the Bayesian updating formula –

Notation	Description
H_i	Enemy Hypothesis (e.g., chase, patrol, random)
O_t	Observed enemy action at time t

4.5 – Enemy Modeling Assumption

ARA++ assumes enemies not only follows deterministic rules (e.g., patrol paths) but also probabilistic strategies (e.g., random moves with preference). Enemy states are known from game rules or models.

- Deterministic Case: The position of enemy e at time t is exactly predictable
- Probabilistic Case: The enemy transitions modeled with probabilities and updated via Bayesian inference
- Multi- enemy Case: The risks are approximated using independent probabilities or Monte Carlo sampling

4.6 – Generality of the Algorithm

The core idea of the algorithm includes the concepts of graph-based searching, enemy and risk which are not limited to a single domain. These domain independent concepts make the algorithm more generalizable so that apart from game playing these algorithms can adapt to vast variety of real-world problems.

- Node can be a location in a maze, state in a game, road in a city, or a network node in cybersecurity
- Enemy can be a guard, traffic obstacle, hazard or attacker
- Risk in every field is a measure of probability of interception or failure

So, extending the boundaries beyond maze problem and game playing, ARA++ is an entire generalized framework.

4.7 – Pseudocode of the Algorithm

Input: start node s, goal node g, enemy model E
Output: risk-aware path from s to g

```
1: initialize open_list with start node s
2: while open_list not empty:
3:     n = node from open_list with lowest f (n, t)
4:     if n == goal:
5:         return path to n
6:     for each neighbor n' of n:
7:         g(n') = g(n) + cost (n → n')
```

8: h(n') = heuristic (n', g)
9: R (n', t+1) = compute_risk (n', t+1, E)
10: f(n') = g(n') + h(n') + λ*R (n', t+1) + μ*D(n')
11: add/update n' in open_list
12: end while

5. Case Study

5.1 Maze Problem Decision Taking Demonstration

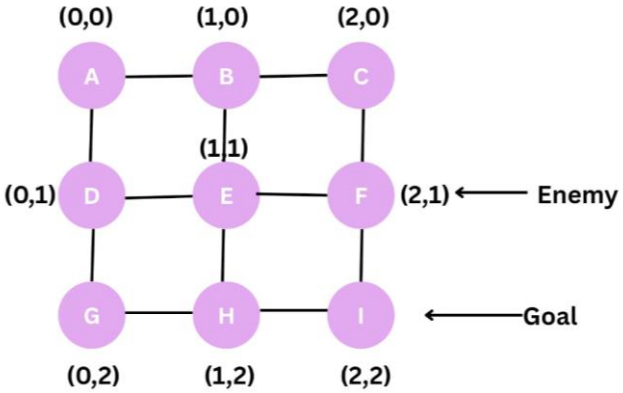

Fig. 5.1 – Maze Problem

Consider the maze problem in the Fig. 5.1 with starting node A, goal node I and initial enemy node F.

Initially, the weight for risk factor is considered to be 0.8, adaptive risk threshold is considered to be 0.3, chase probability is considered to be 60% (0.6), random probability is considered to be 40% (0.4) and for simplicity, deviation penalty is taken as 0.

At time interval t=0, the enemy move probability distribution, risk calculation for nodes and path selection is depicted below -

5.1.1 Probability Distribution

Table 5.1 – Probability Distribution Table

Enemy's Move	Initial distance between enemy and player	Updated distance between enemy and player	Improvement	Chase Probability	Random Probability	Total Probability
F -> E	3	2	+1	0.6/2 = 0.3	0.4/4 = 0.1	0.4
F -> C	3	2	+1	0.6/2 = 0.3	0.4/4 = 0.1	0.4
F -> I	3	4	-1	0.0 (no improvement)	0.4/4 = 0.1	0.1
F -> F	3	3	0	0.0 (no improvement)	0.4/4 = 0.1	0.1

5.1.2 Risk Calculation

Player at node has only two possible moves i.e. A -> B and A -> D, thus risk is calculated for nodes B and D.

Risk for node B –

Table 5.2 – Risk Analysis for Node B

Enemy's Move	Probability	Indicator Function (1 if threatens player, else 0)	Threat Value
F -> C	0.4	1 (adjacent to B)	0.4 * 1 = 0.4
F -> E	0.4	1 (adjacent to B)	0.4 * 1 = 0.4
F - > I	0.1	0	0.1 * 0 = 0.0
F -> F	0.1	0	0.1 * 0 = 0.0

Total Threat Value = 0.4 + 0.4 + 0.0 + 0.0 = 0.8
Survival Value = 1 – 0.8 = 0.2
Risk Percentage = 0.8 * 100 = 80%
Survival Percentage = 0.2 * 100 = 20%

Risk for node D –

Table 5.3 – Risk Analysis for Node D

Enemy's Move	Probability	Indicator Function (1 if threatens player, else 0)	Threat Value
F -> C	0.4	0	0.4 * 0 = 0.0
F -> E	0.4	1 (adjacent to B)	0.4 * 1 = 0.4
F - > I	0.1	0	0.1 * 0 = 0.0
F -> F	0.1	0	0.1 * 0 = 0.0

Total Threat Value = 0.4 + 0.0 + 0.0 + 0.0 = 0.4
Survival Value = 1 – 0.4 = 0.6
Risk Percentage = 0.4 * 100 = 40%
Survival Percentage = 0.6 * 100 = 60%

5.1.3 Selection of Path

Table 5.4 – Path Selection Table

Path	f(n)	h(n)	Λ	R(n,0)	g(n)	Survival Probability
A -> B	1	3	0.8	0.8	1+3+0.8*0.8 = 4.64	1 – 0.8 = 0.2
A -> D	1	3	0.8	0.4	1+3+0.8*0.4 = 4.32	1 – 0.4 = 0.6

As g(n) for path A->D is lesser than that of A->B, it is chosen as more feasible path. Further, as the survival probability of A->B is less than the adaptive risk threshold, the path A -> B is pruned.

Further at time interval t=1, the decisions will be taken according to the updated position of enemy leading to the safer path.

6. Applications of ARA++ Algorithm

Table 6.1 – Applications of ARA++ Algorithm

Domain	Node Representation	Enemy / Adversary	Risk Interpretation	Application of ARA++
Grid/Video Games	Map cells, waypoints	Game guards, AI opponents	Probability of interception	Safer pathfinding while playing tactical RPG, stealth, RTS, dungeon games
Robotics	States of physical environments	Humans, moving robots, obstacles	Collision or mission failure	Search-and-rescue robots avoiding danger while reaching survivors
Autonomous Vehicles	Road segments or multiple areas	Other vehicles, pedestrians	Accident/collision risk	Safe route planning based on time and probability of an accident
Military & Defense	Terrain nodes, air corridors	Hostile drones, enemy patrols	Detection, interception, attack probability	A drone navigating a hostile environment targeting stealth paths or avoiding enemies
Healthcare Robotics	Hospital corridors, spaces or rooms	Patients, staff, equipment	Risk of collision or disruption	Autonomous delivery robots navigate safely with limited crowd movement
Networking & Cybersecurity	Routers, servers, data nodes	Malicious actors, compromised nodes	Probability of data interception	Secure routing of data packets along the network factoring in latency and possible actor in network

7. Results and Discussions

Table 7.1 – Comparison of A*, Risk Aware A* and ARA++

Algorithm	Optimality	Handles Dynamics	Enemy Modeling	Computation	Safety
A*	High	No	None	Low	Low
Risk-Aware A*	Medium	Limited	Static Risk	High	Medium
ARA++ (Proposed)	High	Yes	Bayesian Probabilistic	Moderate (Incremental)	High

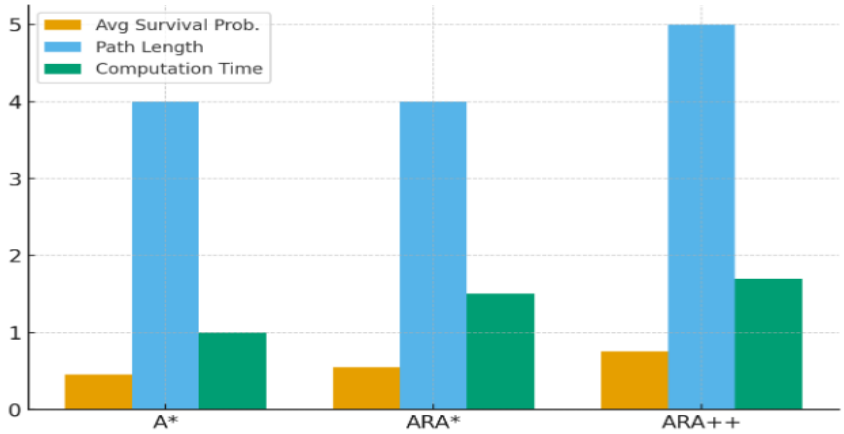

Fig. 7.1 Comparison of Algorithms

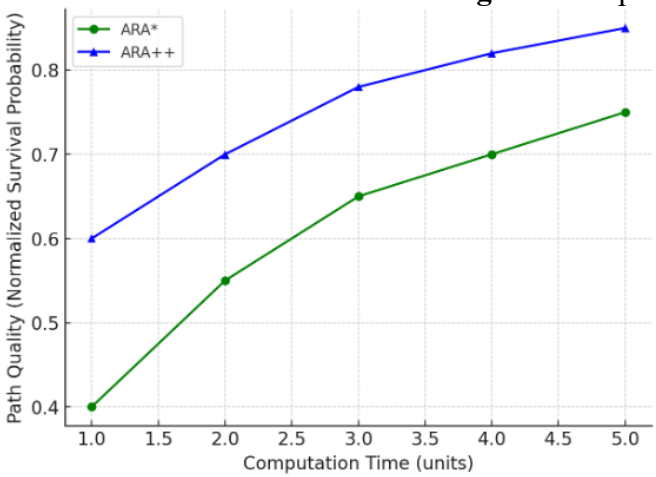

Figure 7.2: Path Quality vs Computation Time

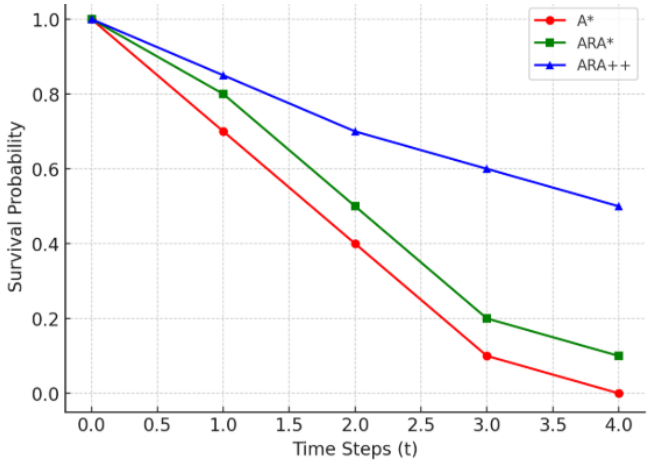

Figure 7.3: Survival Probability vs Time

8. Conclusion

The research introduced the ARA++ algorithm that aims to find the survival critical shortest path in the adversarial environments. Integration of Bayesian enemy prediction. multi-objective optimization and adaptive threshold-based pruning, ARA++ bridged the gap between the shortest path optimization of A* and the survival aware navigation of Risk Aware A*. Thus, the algorithm not only provides computational efficiency but also ensures mission survivability making it a good proposal for real world deployment in uncertain environments. Though the current study demonstrated ARA++ on a simplified maze problem, its potential can be demonstrated and utilized in different sectors including robotics, military and defense and smart transportation. By combining anytime adaptability with survival aware intelligence, ARA++ opens up directions for research and practical deployments in AI driven environments demanding safe navigation with minimum cost under uncertain environments.

References:

1. Pame, Y. G., Kottawar, V. G., & Mahajan, Y. V. (2023, March). A Novel Approach to Maze Solving Algorithm. In 2023 International Conference on Emerging Smart Computing and Informatics (ESCI) (pp. 1-6). IEEE.
2. Alamri, S., Alshehri, S., Alshehri, W., Alamri, H., Alaklabi, A., & Alhmiedat, T. (2021). Autonomous maze solving robotics: Algorithms and systems. International Journal of Mechanical Engineering and Robotics Research, 10(12), 668-675.
3. Chan, K. H., Mustapha, A., & Lee, S. C. (2024). Shortest Pathfinding in a Standard Rectangular Maze using A* Search Algorithm. International Journal of Integrated Engineering, 16(3), 244-256.
4. Martelli, A. (1977). On the complexity of admissible search algorithms. Artificial Intelligence, 8(1), 1-13.

5. Shevtekar, S., Malpe, M., & Bhaila, M. (2022). Analysis of game tree search algorithms using minimax algorithm and alpha-beta pruning. International Journal of Scientific Research in Computer Science, Engineering, and Information Technology (IJSRCSEIT), ISSN, 2456-3307.
6. Parashar, A., Jha, A. K., & Kumar, M. (2022). Analyzing a chess engine based on alpha–beta pruning, enhanced with iterative deepening. In Expert Clouds and Applications: Proceedings of ICOECA 2022 (pp. 691-700). Singapore: Springer Nature Singapore.
7. Wang, J., Liu, M., & Zhao, G. (2024, May). Design of military chess system based on alpha-beta pruning algorithm. In 2024 36th Chinese Control and Decision Conference (CCDC) (pp. 3092-3097). IEEE.
8. Li, J., Zanuttini, B., Cazenave, T., & Ventos, V. (2022). Generalisation of alpha-beta search for AND-OR graphs with partially ordered values (Doctoral dissertation, GREYC CNRS UMR 6072, Universite de Caen).
9. Tran, M. T. (2025). MIN MAX ALPHABETA PRUNING–TIC TAC TOE DEMO. International Journal of Innovation Studies, 9(1), 724-732.
10. Nair, R. R., Babu, T., Kishore, S., Pavithra, K., & Sumana, S. G. (2025, April). Improving Alpha-Beta Pruning Efficiency in Chess Engines. In 2024 International Conference on IT Innovation and Knowledge Discovery (ITIKD) (pp. 1-6). IEEE.
11. Raut, R., Chaudhari, M., Raipalli, S., & Banginwar, S. (2025, February). Enhancing AI Strategy in Checkers through Minimax and Alpha-Beta Pruning Techniques. In 2025 International Conference on Sustainable Energy Technologies and Computational Intelligence (SETCOM) (pp. 1-6). IEEE.
12. Zou, Y. (2021, September). General Pacman AI: Game agent with tree search, adversarial search and model-based RL algorithms. In 2021 2nd International Conference on Big Data & Artificial Intelligence & Software Engineering (ICBASE) (pp. 253-260). IEEE.
13. Novikov, A., & Yanovskyi, V. (2024). Analysis of the decision-making algorithm efficiency in complex game environments on the example of Pac-Man. Information Technologies and Computer Engineering, 3(21), 108-118.
14. Tilkidagi, S. (2021). Monte-Carlo Tree Search Algorithm in Pac-Man Identification of commonalities in 2D video games for realisation in AI (Artificial Intelligence) (Doctoral dissertation, University of Essex).
15. Biju, A., Gayathri, M. P., Jayapandian, N., Chris, A., & Thaleeparambil, N. R. (2025, March). Efficient Pathfinding in a Maze to overcome Challenges in Robotics and AI Using Breadth-First Search. In 2025 IEEE 14th International Conference on Communication Systems and Network Technologies (CSNT) (pp. 727-732). IEEE.
16. Adhikary, S., Agrawal, S., Dave, P., Kothari, S., & Rajpurohit, J. (2024, December). Escape the Maze—An AI-Enabled Game to Create Smart Maze. In International Conference on Soft Computing: Theories and Applications (pp. 487-496). Singapore: Springer Nature Singapore.
17. Bagad, P., Dahatonde, P., Gagare, R., Athare, I., & Ghule, G. (2024, December). Optimizing Pathfinding in Dynamic Environments: A Comparative Study of A* and D-Lite Algorithms. In 2024 IEEE Pune Section International Conference (PuneCon) (pp. 1-11). IEEE.
18. Li, X., Lu, Y., Zhao, X., Deng, X., & Xie, Z. (2024). Path planning for intelligent vehicles based on improved D* Lite. The Journal of Supercomputing, 80(1), 1294-1330.
19. Abdallah, M. A. (2025). Advancing Decision-Making in AI Through Bayesian Inference and Probabilistic Graphical Models. Symmetry, 17(5), 635.
20. Zhang, X., Chan, F. T., Yan, C., & Bose, I. (2022). Towards risk-aware artificial intelligence and machine learning systems: An overview. Decision Support Systems, 159, 113800.
21. Babu, T., Tejaswi, S., Anup, S. B., Reddy, N. R. K., Manasvi, S., & Singh, D. (2025, June). Monte Carlo Tree Search Optimization for Go Game AI. In 2025 International Conference on Emerging Technologies in Computing and Communication (ETCC) (pp. 1-6). IEEE.
22. Sarbini, R. N., Ahmad, I. R. D. A. M., Bura, R. O., & Simbolon, L. U. H. U. T. (2024). Development of pathfinding using a-star and d-star lite algorithms in video game. Journal of Theoretical and Applied Information Technology, 102(3), 832-840.
23. Chatzisavvas, A., Dossis, M., & Dasygenis, M. (2024). Optimizing mobile robot navigation based on A-star algorithm for obstacle avoidance in smart agriculture. Electronics, 13(11), 2057.
24. Mohanty, L., Kumar, A., Mehta, V., Agarwal, M., & Suri, J. S. (2025). Pruning techniques for artificial intelligence networks: a deeper look at their engineering design and bias: the first review of its kind. Multimedia Tools and Applications, 84(11), 9591-9665.
25. Tran, M. T. (2025). MIN MAX ALPHABETA PRUNING–TIC TAC TOE DEMO. International Journal of Innovation Studies, 9(1), 724-732.
26. Boczka, B., Pluzsik, M., Mocsai, T., Botzheim, J., & Reményi, I. (2025, February). Application of a Deep Learning-Based Heuristic for Weighted A-Star Algorithm. In 2025 11th International Conference on Automation, Robotics, and Applications (ICARA) (pp. 145-149). IEEE.

27. Wilkens, I., & El Khoury, J. (2024). Minimax vs. Monte Carlo: A Comparative Case Study on the Use of Minimax with Alpha-Beta Pruning versus Monte Carlo Tree Search as Decision-Making Algorithms in an Android Chess Application.
28. Shevtekar, S., Malpe, M., & Bhaila, M. (2022). Analysis of game tree search algorithms using minimax algorithm and alpha-beta pruning. International Journal of Scientific Research in Computer Science, Engineering, and Information Technology (IJSRCSEIT), ISSN, 2456-3307.
29. Chaudhary, S., Pandey, S., & Khera, M. H. Alpha-Beta Pruning in Mini-Max Algorithm–An Optimized Approach for a Connect-4 Game.